

2026-05-06-p1-f18-no-flicker-rendering

P1-F18 — No-Flicker Rendering Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Goal: Ship a real, end-to-end **no-flicker terminal renderer** for the HelixCode CLI agent. F18 adds an `internal/render/` package with a `Renderer` interface, two impls (`ansiRenderer` for TTY, `plainRenderer` for non-TTY), a `Viewport` for dirty-region tracking, and a `RendererFactory` that selects an impl at startup based on `HELIXCODE_RENDER=plain|fancy|auto` (default `auto`) + `term.IsTerminal(stdout.Fd())`. Streaming LLM tokens flow through `WriteToken` (in-place line update via `\r\033[K`); tool result blocks (LSP diagnostics, sandbox stdout, smart-edit diffs, subagent results) render via `RenderFrame` with dirty-region diff (only changed lines redrawn via cursor positioning + clear-line). Non-TTY fallback is structural: `plainRenderer` emits zero ANSI escapes and zero `\r` characters. **No new external dependencies** — pure `stdlib` + `golang.org/x/term` (already in `go.mod` as indirect at v0.41.0; F18 promotes to direct). **No slash command. No cobra subcommand.** Env var only (`Q5=B`). The renderer is wired into `cmd/cli/main.go::handleGenerate` (lines 1080–1090; existing `fmt.Printf("%s", chunk.Content)` loop replaced with `Begin/WriteToken/Commit`). The agent loop's non-streaming `Generate` path (`internal/agent/base_agent.go::executeTaskWithLLM`) is NOT modified — agent-loop streaming is out of scope for F18.

Architecture: New `internal/render/` package with `types.go` (`Renderer` interface + `Frame` + `RenderMode` + error sentinels + env-var constants), `ansi_renderer.go` (TTY-mode impl; `\r\033[K` for in-place line update; `\033[<n>A\033[<n>B` cursor-positioning for multi-line frame; `\033[?25l\033[?25h` cursor hide/show; per-block `Viewport` for dirty-region diff), `plain_renderer.go` (line-by-line `fmt.Fprint` fallback; per-block buffer flushed at newline boundaries; strips `\r` from tokens silently for log-grep-safety; passes ANSI through verbatim per §5.4), `viewport.go` (`Viewport{lines, lastLines} + Diff(newFrame Frame) []int` returning indices of changed lines; pure Go), `factory.go` (`NewRenderer(opts Options) (Renderer, RenderMode, error)`; reads `HELIXCODE_RENDER` env var via injectable `Options.Env`; probes TTY via `term.IsTerminal` with `Options.IsTTY` test seam; returns `ErrInvalidEnvMode` for typo'd values which `cmd/cli/main.go` falls back to plain on), `tool_helpers.go` (`RenderToolResult(r, id, result)` glue: type-switch over LSP diagnostic / sandbox result / smart-edit result / subagent result; unknown types fall back to `fmt.Fprintln`). Two existing files get tiny additions: `cmd/cli/main.go` (renderer construction at startup + `defer renderer.Close()` + replace streaming-print loop in `handleGenerate` + wrap tool-result printing through `RenderToolResult`).

Tech Stack: Go 1.26, testify v1.11, zap (already in `go.mod`) — already present. **NO new external deps.** `golang.org/x/term` is at v0.41.0 in `go.sum` (verified via `grep "golang.org/x/term" helix_code/go.sum`); currently indirect. After T02 introduces a direct import (`import "golang.org/x/term" in factory.go`), `go mod tidy` promotes the `go.mod` line from `// indirect` to a direct `require` — that is the one expected

`go.mod` change for F18, and `go.sum` is unchanged. Brief justification: (1) ANSI control is a small finite set of byte sequences → pure constants + `fmt.Sprintf` for the parameterised ones; (2) dirty-region diff is a `for i := range max(len(old), len(new))` loop; (3) TTY detection is a single `term.IsTerminal(int)` call from `golang.org/x/term`; (4) width probe (one-shot, factory time) is `term.GetSize(int)`. `go mod tidy` after T02 must produce **zero new entries in `go.sum`** (only the indirect→direct promotion in `go.mod`). T10's verification step asserts this loudly.

Spec: `docs/superpowers/specs/2026-05-06-p1-f18-no-flicker-rendering-design.md` (commit 7f52a9c)

Working directory for go commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F18 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
internal/render && echo BLUFF || echo clean
```

Must always print `clean`.

Anti-bluff hot zone: §5.2 of the spec — F18 can degenerate in four ways: (a) renderer initialised in `main()` but never called from the streaming hot path (the loop in `handleGenerate` still uses `fmt.Printf`); (b) test asserts ANSI sequences in some buffer but never observes the ACTUAL production code path; (c) `plainRenderer` claims fallback but accidentally emits `\r` that breaks log capture (`grep` against the captured log returns mangled lines); (d) dirty-region diff claims to skip unchanged lines but actually re-emits everything (`Viewport.Diff` returns all indices, OR the renderer ignores the diff result). The four “what counts as no-flicker” criteria — (1) production hot path actually goes through `WriteToken/RenderFrame`; (2) `plainRenderer` emits zero `0x1b` and zero `0x0d` bytes; (3) one-line-changed render emits exactly one `\033[<n>A` cursor-up + one `\r\033[K` clear-line; (4) all-lines-changed render emits all dirty-line emissions (negative control) — are each tested with both unit assertions AND a Challenge phase. The Challenge harness uses positive byte evidence (specific byte sequences, byte counts, signature presence). Disk-state mismatch (er, byte-signature mismatch) is a hard Challenge failure. Absence-of-error is NEVER acceptable.

Why this is consequential: flicker-free streaming is the difference between a CLI that *feels* like an interactive agent and a CLI that *feels* like a print-line script. Claude-code's perceived polish vs other open-source agents largely traces to in-place updates done right. F18's discriminating tests are: (i) the Challenge's STREAMING-FANCY phase (assert capture contains 10 `\r\0x1b[K` sequences for 10 streamed tokens — proving the production hot path goes through the renderer, NOT through `fmt.Printf`); (ii) the Challenge's STREAMING-PLAIN phase (capture contains zero `\0x1b` and zero `\0x0d` — proving the non-TTY fallback is safe to `grep`); (iii) the Challenge's DIRTY-REGION-DIFF phase (one-line-change capture is strictly smaller than initial render AND contains exactly one cursor-up sequence — proving the diff actually saves bytes). All three must produce positive evidence; none can be satisfied by absence-of-error.

Task list



P1-F18-T01 — bootstrap evidence + advance PROGRESS to F18



P1-F18-T02 — `internal/render/types.go`: `Renderer` interface + `RenderMode` enum + `Frame` + error sentinels + env-var constants (TDD)



P1-F18-T03 — `internal/render/ansi_renderer.go`: in-place line update via `\r\033[K` + multi-line frame rendering with dirty-region diff (TDD against `bytes.Buffer`)



P1-F18-T04 — `internal/render/plain_renderer.go`: line-by-line `fmt.Fprint` fallback with newline-boundary buffering + `\r-strip` + zero-ANSI invariant (TDD)



P1-F18-T05 — `internal/render/viewport.go`: `Frame` buffer + dirty-line tracking + pure-Go Diff (TDD)



P1-F18-T06 — `internal/render/factory.go`: `RendererFactory` with `HELIXCODE_RENDER` env var + TTY detection via `golang.org/x/term`; constructor-injection seams for tests (TDD)



P1-F18-T07 — Wire LLM streaming hook in `cmd/cli/main.go::handleGenerate` (replace `fmt.Printf("%s ", chunk.Content)` loop with `Begin/WriteToken/Commit`) (TDD)



P1-F18-T08 — `internal/render/tool_helpers.go` + wire tool-result frame rendering at the existing print sites in `cmd/cli/main.go` (TDD)



P1-F18-T09 — Challenge harness (4 always-run phases + 1 TTY-gated phase: `STREAMING-FANCY` + `STREAMING-PLAIN` + `DIRTY-REGION-DIFF` + `TTY-FALLBACK` + `REAL-TTY`) with positive byte evidence



P1-F18-T10 — Feature 18 close-out + push 4 remotes non-force

Task 1: Bootstrap

Append F18 evidence section header (spec 7f52a9c), update PROGRESS current focus to F18 (replacing the F17 close-out's "F18 next candidate" pointer), insert F18 task list (10 items) after F17's. Confirm `06_phase_1_evidence.md` has an F18 anchor. Verify `golang.org/x/term v0.41.0` is in `helix_code/go.sum` (sanity check before T06 promotes it to direct).

Commit: docs(P1-F18-T01): bootstrap Phase 1 / Feature 18 evidence + advance PROGRESS.

Task 2: types.go (TDD)

Files: new `helix_code/internal/render/types.go`, new `helix_code/internal/render/types_test.go`.

Define: - `RenderMode` enum (`RenderModePlain = 0`, `RenderModeFancy = 1`) with `String()` string method ("plain" / "fancy"). - `Frame`{`ID` string; `Lines`

[]string}. - Renderer interface (Begin / WriteToken / RenderFrame / Commit / Close / Mode). - Error sentinels (ErrUnknownBlockID, ErrInvalidEnvMode). - Env-var constants (EnvRenderMode = "HELIXCODE_RENDER", ModePlainEnv = "plain", ModeFancyEnv = "fancy", ModeAutoEnv = "auto").

Failing tests FIRST:

```
func TestRenderMode_String_Plain(t *testing.T) {
    require.Equal(t, "plain", RenderModePlain.String())
}
func TestRenderMode_String_Fancy(t *testing.T) {
    require.Equal(t, "fancy", RenderModeFancy.String())
}

func TestErrorSentinels_DistinctErrorsIs(t *testing.T) {
    for _, e := range []error{ErrUnknownBlockID,
        ErrInvalidEnvMode} {
        wrapped := fmt.Errorf("wrapped: %w", e)
        require.ErrorIs(t, wrapped, e)
    }
}

func TestEnvVarConstants_ExactValues(t *testing.T) {
    require.Equal(t, "HELIXCODE_RENDER", EnvRenderMode)
    require.Equal(t, "plain", ModePlainEnv)
    require.Equal(t, "fancy", ModeFancyEnv)
    require.Equal(t, "auto", ModeAutoEnv)
}

func TestFrame_FieldsZeroValueOK(t *testing.T) {
    f := Frame{}
    require.Empty(t, f.ID)
    require.Nil(t, f.Lines)
}
```

Subject: feat(P1-F18-T02): Renderer interface + RenderMode enum + Frame + error sentinels.

Task 3: ansi_renderer.go (TDD)

Files: new helix_code/internal/render/ansi_renderer.go, new helix_code/internal/render/ansi_renderer_test.go.

ansi_renderer.go exports nothing public (the factory returns the Renderer interface). Internals:

```
type ansiRenderer struct {
    w      io.Writer
    width  int
```

```

    mu    sync.Mutex
    blocks map[string]*blockState
    cursorHidden bool
}
type blockState struct {
    viewport    *Viewport
    activeLine  string
    activeLineIdx int
    inProgress  bool
}
func newAnsiRenderer(w io.Writer, width int) *ansiRenderer
// implements Renderer per spec §3.3

```

Key implementation points (spec §4.2 + §4.3): - Begin: lazy block creation; emit `\033[?25l` (hide cursor) on FIRST Begin call. - WriteToken: append to `bs.activeLine`; on newline, flush completed lines with `\r\x1b[K%s\n`; emit current partial via `\r\x1b[K%s`. - RenderFrame: first frame → write all lines + `\n`; subsequent frames → `Viewport.Diff` → for each dirty index, emit `\x1b[<n>A\r\x1b[K<line>\x1b[<n>B\r`; then `Viewport.Apply(frame)`. - Commit: emit final `\n`; mark block as not-in-progress. - Close: commit any open blocks; emit `\033[?25h` (show cursor) if hidden. - Mode: returns `RenderModeFancy`.

Failing tests FIRST:

```

func TestAnsiRenderer_WriteToken_EmitsClearLineSequence(t
    *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    r.WriteToken("b1", "hi")
    // expected: \x1b[?25l + \r\x1b[Khi
    got := buf.String()
    require.Contains(t, got, "\x1b[?25l")
    require.Contains(t, got, "\r\x1b[Khi")
}

func TestAnsiRenderer_WriteToken_TwoTokens_AccumulateAndRedraw(t
    *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    r.WriteToken("b1", "hello")
    r.WriteToken("b1", "world")
    got := buf.String()
    // First WriteToken emits \r\x1b[Khello; second emits
    // \r\x1b[Khelloworld
    require.Contains(t, got, "\r\x1b[Khello")
    require.Contains(t, got, "\r\x1b[Khelloworld")
}

```

```

func TestAnsiRenderer_WriteToken_NewlineCommitsLine(t
    *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    r.WriteToken("b1", "line1\n")
    got := buf.String()
    require.Contains(t, got, "\r\x1b[Kline1\n")
}

func TestAnsiRenderer_RenderFrame_FirstFrame_EmitsAllLines(t
    *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    r.RenderFrame("b1", Frame{ID: "b1", Lines: []string{"a",
        "b", "c"}})
    got := buf.String()
    require.Contains(t, got, "a\n")
    require.Contains(t, got, "b\n")
    require.Contains(t, got, "c\n")
}

func TestAnsiRenderer_RenderFrame_OneLineChanged_EmitsOneCursorUp_DirtyReg
    *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    r.RenderFrame("b1", Frame{ID: "b1", Lines: []string{"a",
        "b", "c", "d", "e"}})
    firstLen := buf.Len()
    buf.Reset()
    r.RenderFrame("b1", Frame{ID: "b1", Lines: []string{"a",
        "b", "CHANGED", "d", "e"}})
    secondLen := buf.Len()
    got := buf.String()
    // CRITICAL anti-bluff: second render < first render (proves
        dirty-region diff)
    require.Less(t, secondLen, firstLen)
    // CRITICAL anti-bluff: exactly ONE cursor-up sequence
    require.Equal(t, 1, strings.Count(got, "\x1b[")) //
        approximation; tighten in impl
    require.Contains(t, got, "\r\x1b[K")
    require.Contains(t, got, "CHANGED")
    require.NotContains(t, got, "a\n") // unchanged lines NOT
        re-emitted
    _ = firstLen
}

```

```

func TestAnsiRenderer_RenderFrame_AllLinesChanged_EmitsAllDirtyEmissions(t
    *testing.T) {
    // Negative control: when all lines change, all 5 should be
    // re-emitted via dirty-region writes
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    r.RenderFrame("b1", Frame{ID: "b1", Lines: []string{"a",
        "b", "c", "d", "e"}})
    buf.Reset()
    r.RenderFrame("b1", Frame{ID: "b1", Lines: []string{"A",
        "B", "C", "D", "E"}})
    got := buf.String()
    require.Equal(t, 5, strings.Count(got, "\r\x1b[K"))
}

func TestAnsiRenderer_Commit_EmitsTrailingNewline(t *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    r.WriteToken("b1", "x")
    buf.Reset()
    r.Commit("b1")
    require.Contains(t, buf.String(), "\n")
}

func TestAnsiRenderer_Close_RestoresCursor(t *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    require.NoError(t, r.Close())
    require.Contains(t, buf.String(), "\x1b[?25h")
}

func TestAnsiRenderer_TokenContainsAnsiEscape_PassedThroughVerbatim(t
    *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    r.Begin("b1")
    r.WriteToken("b1", "\x1b[31mred\x1b[0m")
    require.Contains(t, buf.String(), "\x1b[31mred\x1b[0m")
}

func TestAnsiRenderer_ConcurrentBlocks_SerialisedNoPanic(t
    *testing.T) {
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)

```

```

var wg sync.WaitGroup
for i := 0; i < 4; i++ {
    wg.Add(1)
    go func(i int) {
        defer wg.Done()
        id := fmt.Sprintf("b%d", i)
        r.Begin(id)
        r.WriteToken(id, fmt.Sprintf("token-%d\n", i))
        r.Commit(id)
    }(i)
}
wg.Wait() // no panic = pass
}

```

Subject: feat(P1-F18-T03): ansiRenderer with in-place line updates + dirty-region frame diff.

Task 4: plain_renderer.go (TDD)

Files: new helix_code/internal/render/plain_renderer.go, new helix_code/internal/render/plain_renderer_test.go.

plain_renderer.go:

```

type plainRenderer struct {
    w io.Writer
    mu sync.Mutex
    buffers map[string]*strings.Builder
}
func newPlainRenderer(w io.Writer) *plainRenderer
// implements Renderer per spec §3.3

```

Key implementation points (spec §4.2 + §5.1): - Begin: lazy block creation (empty strings.Builder). - WriteToken: strip \r silently from text; append to per-block buffer; flush whole lines (every \n boundary) with fmt.Fprintln. - RenderFrame: print each line via fmt.Fprintln. - Commit: flush remaining buffer with trailing \n. - Close: flush all open blocks. - Mode: returns RenderModePlain.

Critical invariants (anti-bluff §5.2 (c)): - Captured output contains zero 0x1b bytes (no ANSI). - Captured output contains zero 0x0d bytes (no CR).

Failing tests FIRST:

```

func TestPlainRenderer_WriteToken_PartialLineBufferedUntilNewline(t
    *testing.T) {
    var buf bytes.Buffer
    r := newPlainRenderer(&buf)
    r.Begin("b1")
    r.WriteToken("b1", "hello")
}

```



```

        require.Empty(t, buf.String()) // partial line not flushed
        yet
    }

    func TestPlainRenderer_WriteToken_FullLineFlushed(t *testing.T) {
        var buf bytes.Buffer
        r := newPlainRenderer(&buf)
        r.Begin("b1")
        r.WriteToken("b1", "hello\n")
        require.Equal(t, "hello\n", buf.String())
    }

    func TestPlainRenderer_RenderFrame_PrintsLines(t *testing.T) {
        var buf bytes.Buffer
        r := newPlainRenderer(&buf)
        r.Begin("b1")
        r.RenderFrame("b1", Frame{ID: "b1", Lines: []string{"a",
            "b", "c"}})
        require.Equal(t, "a\nb\nc\n", buf.String())
    }

    func TestPlainRenderer_NoAnsiEscapesEver(t *testing.T) {
        var buf bytes.Buffer
        r := newPlainRenderer(&buf)
        r.Begin("b1")
        for i := 0; i < 100; i++ {
            r.WriteToken("b1", fmt.Sprintf("token-%d ", i))
        }
        r.WriteToken("b1", "\n")
        r.RenderFrame("b1", Frame{ID: "b1", Lines: []string{"l1",
            "l2", "l3", "l4", "l5"}})
        r.Commit("b1")
        got := buf.Bytes()
        require.NotContains(t, got, byte(0x1b), "plain renderer must
            never emit ANSI ESC")
    }

    func TestPlainRenderer_NoCarriageReturnsEver(t *testing.T) {
        var buf bytes.Buffer
        r := newPlainRenderer(&buf)
        r.Begin("b1")
        for i := 0; i < 100; i++ {
            r.WriteToken("b1", fmt.Sprintf("tok%d\n", i))
        }
        r.RenderFrame("b1", Frame{ID: "b1", Lines: []string{"a",
            "b"}})
        r.Commit("b1")
        got := buf.Bytes()
    }

```

```

        require.NotContains(t, got, byte(0x0d), "plain renderer must
            never emit \\r")
    }

func TestPlainRenderer_TokenContainsCR_StrippedSilently(t
    *testing.T) {
    var buf bytes.Buffer
    r := newPlainRenderer(&buf)
    r.Begin("b1")
    r.WriteToken("b1", "hello\\rworld\\n")
    require.Equal(t, "helloworld\\n", buf.String())
}

func TestPlainRenderer_TokenContainsAnsi_PassedThroughVerbatim(t
    *testing.T) {
    // §5.4 v1 policy: pass-through ANSI from user-provided text
    // (asymmetric vs CR strip).
    var buf bytes.Buffer
    r := newPlainRenderer(&buf)
    r.Begin("b1")
    r.WriteToken("b1", "\\x1b[31mred\\x1b[0m\\n")
    require.Equal(t, "\\x1b[31mred\\x1b[0m\\n", buf.String())
}

func TestPlainRenderer_Commit_FlushesPendingBuffer(t *testing.T)
{
    var buf bytes.Buffer
    r := newPlainRenderer(&buf)
    r.Begin("b1")
    r.WriteToken("b1", "no-newline-yet")
    r.Commit("b1")
    require.Equal(t, "no-newline-yet\\n", buf.String())
}

func TestPlainRenderer_Close_FlushesAllBlocks(t *testing.T) { /*
    multi-block; Close flushes all */ }

func TestPlainRenderer_Mode_IsPlain(t *testing.T) {
    r := newPlainRenderer(&bytes.Buffer{})
    require.Equal(t, RenderModePlain, r.Mode())
}

```

Subject: feat(P1-F18-T04): plainRenderer with line-by-line fallback
+ zero-ANSI/zero-CR invariants.

Task 5: viewport.go (TDD)

Files: new helix_code/internal/render/viewport.go, new helix_code/internal/render/viewport_test.go.

viewport.go:

```
type Viewport struct {
    lines      []string
    lastLines []string
}
func NewViewport() *Viewport
func (v *Viewport) Diff(newFrame Frame) []int
func (v *Viewport) Apply(newFrame Frame)
```

Diff returns 0-indexed line indices that changed. Includes added indices ($i \geq \text{len}(\text{lastLines})$) and removed indices ($i \geq \text{len}(\text{newFrame.Lines})$).

Failing tests FIRST:

```
func TestViewport_Diff_FirstFrame_AllNew(t *testing.T) {
    v := NewViewport()
    dirty := v.Diff(Frame{Lines: []string{"a", "b", "c"}})
    require.Equal(t, []int{0, 1, 2}, dirty)
}

func TestViewport_Diff_NoChange_EmptyDirty(t *testing.T) {
    v := NewViewport()
    v.Apply(Frame{Lines: []string{"a", "b", "c"}})
    dirty := v.Diff(Frame{Lines: []string{"a", "b", "c"}})
    require.Empty(t, dirty)
}

func TestViewport_Diff_OneLineChanged_OneIndex(t *testing.T) {
    v := NewViewport()
    v.Apply(Frame{Lines: []string{"a", "b", "c", "d", "e"}})
    dirty := v.Diff(Frame{Lines: []string{"a", "b", "CHANGED",
                                         "d", "e"}})
    require.Equal(t, []int{2}, dirty)
}

func TestViewport_Diff_AllLinesChanged_AllIndices(t *testing.T) {
    v := NewViewport()
    v.Apply(Frame{Lines: []string{"a", "b", "c"}})
    dirty := v.Diff(Frame{Lines: []string{"A", "B", "C"}})
    require.Equal(t, []int{0, 1, 2}, dirty)
}

func TestViewport_Diff_NewFrameLonger_AddedIndicesIncluded(t
    *testing.T) {
```

```

    v := NewViewport()
    v.Apply(Frame{Lines: []string{"a", "b"}})
    dirty := v.Diff(Frame{Lines: []string{"a", "b", "c", "d"}})
    require.Equal(t, []int{2, 3}, dirty)
}

func TestViewport_Diff_NewFrameShorter_RemovedIndicesIncluded(t
    *testing.T) {
    v := NewViewport()
    v.Apply(Frame{Lines: []string{"a", "b", "c", "d"}})
    dirty := v.Diff(Frame{Lines: []string{"a", "b"}})
    require.Equal(t, []int{2, 3}, dirty)
}

func TestViewport_Apply_UpdatesLastLines(t *testing.T) {
    v := NewViewport()
    v.Apply(Frame{Lines: []string{"x", "y"}})
    require.Equal(t, []string{"x", "y"}, v.lastLines)
}

```

Subject: feat(P1-F18-T05): Viewport with pure-Go dirty-line Diff + Apply.

Task 6: factory.go (TDD)

Files: new helix_code/internal/render/factory.go, new helix_code/internal/render/factory_test.go.

factory.go introduces the direct golang.org/x/term import. After this commit, run go mod tidy once — go.mod line for golang.org/x/term v0.41.0 drops the // indirect marker; go.sum is unchanged. T10 verifies this loudly.

```

import "golang.org/x/term"

type Options struct {
    Writer io.Writer
    IsTTY  *bool
    Env    func(string) string
    Width  int
}

func NewRenderer(opts Options) (Renderer, RenderMode, error) {
    if opts.Writer == nil { opts.Writer = os.Stdout }
    if opts.Env == nil     { opts.Env = os.Getenv }
    env := opts.Env(EnvRenderMode)
    var isTTY bool
    if opts.IsTTY != nil { isTTY = *opts.IsTTY } else { isTTY =
        term.IsTerminal(int(os.Stdout.Fd())) }
    width := opts.Width

```

```

if width == 0 {
    if w, _, err := term.GetSize(int(os.Stdout.Fd())); err
        == nil { width = w } else { width = 80 }
}
switch env {
case ModePlainEnv:
    return newPlainRenderer(opts.Writer, RenderModePlain,
        nil)
case ModeFancyEnv:
    return newAnsiRenderer(opts.Writer, width),
        RenderModeFancy, nil
case ModeAutoEnv, "":
    if isTTY { return newAnsiRenderer(opts.Writer, width),
        RenderModeFancy, nil }
    return newPlainRenderer(opts.Writer, RenderModePlain,
        nil)
default:
    return newPlainRenderer(opts.Writer, RenderModePlain,
        fmt.Errorf("%w: got %q", ErrInvalidEnvMode, env))
}
}

```

Failing tests FIRST (mode-resolution truth table — spec §4.4):

```

func boolPtr(b bool) *bool { return &b }
func envFn(m map[string]string) func(string) string {
    return func(k string) string { return m[k] }
}

func TestFactory_AutoTTY_Fancy(t *testing.T) {
    r, mode, err := NewRenderer(Options{
        Writer: &bytes.Buffer{},
        IsTTY:   boolPtr(true),
        Env:     envFn(map[string]string{"HELIXCODE_RENDER":
            "auto"}),
    })
    require.NoError(t, err)
    require.Equal(t, RenderModeFancy, mode)
    require.Equal(t, RenderModeFancy, r.Mode())
}

func TestFactory_AutoNonTTY_Plain(t *testing.T) {
    _, mode, err := NewRenderer(Options{
        Writer: &bytes.Buffer{},
        IsTTY:   boolPtr(false),
        Env:     envFn(map[string]string{"HELIXCODE_RENDER":
            "auto"}),
    })
    require.NoError(t, err)
}

```

```

    require.Equal(t, RenderModePlain, mode)
}

func TestFactory_Unset_TreatedAsAuto_TTYTrue_Fancy(t *testing.T)
{
    _, mode, err := NewRenderer(Options{IsTTY: boolPtr(true),
        Env: envFn(map[string]string{}), Writer:
            &bytes.Buffer{}})
    require.NoError(t, err)
    require.Equal(t, RenderModeFancy, mode)
}

func TestFactory_Unset_TreatedAsAuto_TTYFalse_Plain(t
    *testing.T) {
    _, mode, err := NewRenderer(Options{IsTTY: boolPtr(false),
        Env: envFn(map[string]string{}), Writer:
            &bytes.Buffer{}})
    require.NoError(t, err)
    require.Equal(t, RenderModePlain, mode)
}

func TestFactory_PlainTTY_Plain(t *testing.T) {
    _, mode, _ := NewRenderer(Options{IsTTY: boolPtr(true), Env:
        envFn(map[string]string{"HELIXCODE_RENDER": "plain"}),
        Writer: &bytes.Buffer{}})
    require.Equal(t, RenderModePlain, mode)
}

func TestFactory_FancyNonTTY_Fancy_UserOptIn(t *testing.T) {
    _, mode, _ := NewRenderer(Options{IsTTY: boolPtr(false),
        Env: envFn(map[string]string{"HELIXCODE_RENDER":
            "fancy"}), Writer: &bytes.Buffer{}})
    require.Equal(t, RenderModeFancy, mode)
}

func TestFactory_Garbage_FallbackPlain_ReturnsErr(t *testing.T) {
    r, mode, err := NewRenderer(Options{IsTTY: boolPtr(true),
        Env: envFn(map[string]string{"HELIXCODE_RENDER":
            "garbage"}), Writer: &bytes.Buffer{}})
    require.ErrorIs(t, err, ErrInvalidEnvMode)
    require.Equal(t, RenderModePlain, mode) // fallback
    require.Equal(t, RenderModePlain, r.Mode())
}

func TestFactory_DefaultOptions_UsesOsStdout(t *testing.T) {
    r, _, err := NewRenderer(Options{IsTTY: boolPtr(false), Env:
        envFn(map[string]string{}})
    require.NoError(t, err)

```

```

        require.NoError(t, r.Close())
    }

    func TestNoLoggerInfoOrDebugTakesContent_CONST042_SourceScan(t
        *testing.T) {
        // Anti-bluff §5.2: scan internal/render/*.go for any
        // logger.Info/Debug
        // call site that takes "token" or "frame" or "content" or
        // "text" as an arg.
        // Must be zero matches.
        matches, err := filepath.Glob("*.go")
        require.NoError(t, err)
        forbidden := regexp.MustCompile(`logger\.\b(Info|Debug)
            \b.*\b(token|frame|content|text)\b`)
        for _, f := range matches {
            b, err := os.ReadFile(f); require.NoError(t, err)
            require.False(t, forbidden.Match(b), "%s contains
                forbidden logger call: CONST-042", f)
        }
    }
}

```

Subject: feat(P1-F18-T06): RendererFactory with HELIXCODE_RENDER env var + TTY detection via x/term.

Task 7: Wire LLM streaming hook in cmd/cli/main.go (TDD)

Files: modify `helix_code/cmd/cli/main.go`, modify (or extend) `helix_code/cmd/cli/main_test.go` if it exists, OR create a new integration test in `helix_code/tests/integration/render_test.go`.

`main.go` changes:

1. **Renderer construction at startup** (in `main()` or `NewCLI`):

```

import "dev.helix.code/internal/render"

r, mode, rerr := render.NewRenderer(render.Options{})
if rerr != nil {

    logger.Warn("invalid HELIXCODE_RENDER value; falling
        back to plain", zap.Error(rerr))
}
logger.Info("render mode", zap.String("mode", mode.String()))
defer r.Close()
c.renderer = r

```

Add `renderer render.Renderer` field to the CLI struct.

2. **Replace streaming-print loop in** `handleGenerate` (lines 1080–1090 of current `cmd/cli/main.go`):

```
if stream {
    chunkChan := make(chan llm.LLMResponse, 100)
    err := provider.GenerateStream(ctx, req, chunkChan)
    if err != nil {
        return fmt.Errorf("streaming generation failed: %w",
            err)
    }
    blockID := fmt.Sprintf("llm-stream-%d",
        time.Now().UnixNano())
    c.renderer.Begin(blockID)
    for chunk := range chunkChan {
        c.renderer.WriteToken(blockID, chunk.Content)
    }
    c.renderer.Commit(blockID)
}
```

Failing tests FIRST (in `tests/integration/render_test.go` with `//go:build integration`):

```
//go:build integration
// +build integration
```

```
package integration
```

```
func TestRender_StreamingPipeline_FancyMode_EmitsClearLineSequences(t
    *testing.T) {
    var buf bytes.Buffer
    r, mode, err := render.NewRenderer(render.Options{
        Writer: &buf, IsTTY: ptrTrue(), Env: func(string) string
            { return "" },
    })
    require.NoError(t, err)
    require.Equal(t, render.RenderModeFancy, mode)
    defer r.Close()
    blockID := "test-stream"
    r.Begin(blockID)
    tokens := []string{"Hello", " ", "world", "!"}
    for _, tok := range tokens { r.WriteToken(blockID, tok) }
    r.Commit(blockID)
    got := buf.String()
    // Each token causes an in-place redraw via \r\x1b[K
    require.GreaterOrEqual(t, strings.Count(got, "\r\x1b[K"), 4)
    require.Contains(t, got, "Hello world!")
}
```



```

func TestRender_StreamingPipeline_PlainMode_NoAnsiNoCR(t
    *testing.T) {
    var buf bytes.Buffer
    r, mode, err := render.NewRenderer(render.Options{
        Writer: &buf, IsTTY: ptrFalse(), Env: func(string)
            string { return "" },
    })
    require.NoError(t, err)
    require.Equal(t, render.RenderModePlain, mode)
    defer r.Close()
    r.Begin("p1")
    for _, tok := range []string{"Hello", " ", "world", "!\n"} {
        r.WriteToken("p1", tok) }
    r.Commit("p1")
    got := buf.Bytes()
    require.NotContains(t, got, byte(0x1b))
    require.NotContains(t, got, byte(0x0d))
    require.Contains(t, string(got), "Hello world!")
}

func TestRender_HandleGenerateStreamPath_GoesThroughRendererNotFmtPrint(t
    *testing.T) {
    // Wire up a real CLI with a FakeProvider; capture stdout via
    // the renderer's Writer.
    // Assert the captured output contains the renderer's
    // signature byte pattern,
    // NOT the today's "%s \" (token-with-space) pattern.
    var buf bytes.Buffer
    r, _, _ := render.NewRenderer(render.Options{Writer: &buf,
        IsTTY: ptrTrue(), Env: func(string) string { return
            "" }})
    cli := &CLI{ llmProvider: &fakeProvider{tokens:
        []string{"a", "b", "c"}}, renderer: r }
    err := cli.handleGenerate(context.Background(), "p",
        "fake:m", 100, 0.0, true)
    require.NoError(t, err)
    require.NoError(t, r.Close())
    got := buf.String()
    require.Contains(t, got, "\r\x1b[K")
    // Negative: ensure the today's bluffy "%s " pattern is NOT
    // used (3 tokens with single-space separator)
    require.NotContains(t, got, "a b c ")
}

```

Subject: feat(P1-F18-T07): wire renderer into handleGenerate
streaming hot path + render mode log.

Task 8: tool_helpers.go + wire tool result rendering (TDD)

Files: new helix_code/internal/render/tool_helpers.go, new helix_code/internal/render/tool_helpers_test.go, modify helix_code/cmd/cli/main.go (call RenderToolResult at existing tool-result print sites).

tool_helpers.go:

```
package render

// RenderToolResult inspects known tool-result types and renders
// them as
// a Frame via r.RenderFrame. Unknown types fall back to
// fmt.Fprintln on
// the underlying writer (via r.Begin/RenderFrame with a one-line
// frame).
func RenderToolResult(r Renderer, blockID string, result
    interface{}) {
    frame := convertResultToFrame(blockID, result)
    r.Begin(blockID)
    r.RenderFrame(blockID, frame)
    r.Commit(blockID)
}

func convertResultToFrame(id string, result interface{}) Frame {
    switch v := result.(type) {
    case Frame:
        v.ID = id; return v
    case []string:
        return Frame{ID: id, Lines: v}
    case string:
        return Frame{ID: id, Lines: strings.Split(v, "\n")}
    case fmt.Stringer:
        return Frame{ID: id, Lines: strings.Split(v.String(),
            "\n")}
    default:
        return Frame{ID: id, Lines: []string{fmt.Sprintf("%v",
            result)}}
    }
}
```

The conversion is intentionally generic — the renderer package does NOT import lsp/sandbox/smartedit/subagent types directly (would create import cycles). Those callers can pass an already-formatted []string or a Frame directly. The CLI's tool-result print sites convert their typed results to Frame or []string BEFORE calling RenderToolResult.

Failing tests FIRST (tool_helpers_test.go):

```
func TestRenderToolResult_Frame_PassesThrough(t *testing.T) {
    var buf bytes.Buffer
```

```

    r := newPlainRenderer(&buf)
    RenderToolResult(r, "b1", Frame{Lines: []string{"a", "b",
        "c"}}})
    require.Equal(t, "a\nb\nc\n", buf.String())
}

func TestRenderToolResult_StringSlice_RendersLines(t *testing.T)
{
    var buf bytes.Buffer
    r := newPlainRenderer(&buf)
    RenderToolResult(r, "b1", []string{"x", "y"})
    require.Equal(t, "x\ny\n", buf.String())
}

func TestRenderToolResult_String_SplitsOnNewline(t *testing.T) {
    var buf bytes.Buffer
    r := newPlainRenderer(&buf)
    RenderToolResult(r, "b1", "line1\nline2")
    require.Contains(t, buf.String(), "line1\n")
    require.Contains(t, buf.String(), "line2\n")
}

func TestRenderToolResult_Stringer_UsesString(t *testing.T) {
    var buf bytes.Buffer
    r := newPlainRenderer(&buf)
    RenderToolResult(r, "b1", &fakeStringer{"stringified"})
    require.Contains(t, buf.String(), "stringified")
}

func TestRenderToolResult_Unknown_FallsBackToFmt(t *testing.T) {
    var buf bytes.Buffer
    r := newPlainRenderer(&buf)
    RenderToolResult(r, "b1", 42) // int – not handled specially
    require.Contains(t, buf.String(), "42")
}

func TestRenderToolResult_DirtyDiff_OneLineChanged_FewerBytes(t
    *testing.T) {
    // ANSI mode: render twice with one line changed; second
    // render < first render.
    var buf bytes.Buffer
    r := newAnsiRenderer(&buf, 80)
    RenderToolResult(r, "b1", []string{"a", "b", "c", "d", "e"})
    firstLen := buf.Len()
    buf.Reset()
    RenderToolResult(r, "b1", []string{"a", "b", "CHANGED", "d",
        "e"})
    secondLen := buf.Len()

```

```

    require.Less(t, secondLen, firstLen)
    _ = firstLen
}

```

Wire-in: replace selected `fmt.Println(result) / fmt.Printf("%v\n", result)` call sites in `cmd/cli/main.go` for tool results with `render.RenderToolResult(c.renderer, blockID, result)`. The integration test from T07 is extended to assert the wired path produces the expected byte pattern.

Subject: feat(P1-F18-T08): RenderToolResult helper + wire tool-result frame rendering at CLI print sites.

Task 9: Challenge harness (5-phase, byte-signature positive evidence)

Files: new `helix_code/tests/integration/cmd/p1f18_challenge/main.go`, new `challenges/p1-f18-no-flicker-rendering/CHALLENGE.md`, new `challenges/p1-f18-no-flicker-rendering/run.sh`.

Harness phases (per spec §6.3):

1. **STREAMING-FANCY (always runs)** — construct `ansiRenderer` with `bytes.Buffer` writer + `IsTTY=true`; fake LLM provider emits 10 tokens; assert capture contains 10 `\r\x1b[K` sequences AND ≤ 1 trailing `\n` AND the concatenated token text.
2. **STREAMING-PLAIN (always runs)** — construct `plainRenderer` with `bytes.Buffer` writer + `IsTTY=false`; same 10 tokens; assert capture contains zero `\x1b` AND zero `\x0d` AND the concatenated token text byte-for-byte.
3. **DIRTY-REGION-DIFF (always runs)** — render initial 5-line frame; render same frame with line 2 changed; assert second-render byte count < first-render byte count AND second-render contains exactly 1 `\x1b[<n>A` cursor-up + 1 `\r\x1b[K` clear-line. Negative control: render with all 5 lines changed; assert all 5 dirty-line emissions present.
4. **TTY-FALLBACK (always runs)** — construct renderer via factory with `HELIXCODE_RENDER=auto` + `IsTTY=false`; resolved mode = plain; run streaming + tool-block pipelines; assert capture has zero `\x1b` AND zero `\x0d`.
5. **REAL-TTY (gated; SKIP-OK on non-TTY)** — `term.IsTerminal(int(os.Stdout.Fd()))`; if non-TTY, SKIP-OK with marker SKIP-OK: P1-F18 real-TTY phase requires interactive terminal (CI / pipe). When TTY, construct renderer via factory; assert mode = fancy; stream 10 tokens to `os.Stdout` for real; `Close()` returns nil.

Output skeleton (verbatim per spec §6.3) ends with:

SUMMARY: STREAMING-FANCY=7/7 PASS; STREAMING-PLAIN=7/7 PASS; DIRTY-REGION-

The Challenge MUST exit non-zero on any byte-signature mismatch. Absence-of-error is NEVER acceptable. SHA / size mismatches (e.g., `secondLen >= firstLen` when one line changed) are hard failures. Anti-bluff smoke clean check appended to harness output. Verbatim output captured into `06_phase_1_evidence.md`. Dual commit (Challenges submodule + meta-repo bump).

Subject: feat(P1-F18-T09): challenge with byte-signature positive evidence per phase (4 always-run + 1 TTY-gated).

Task 10: Close-out + push

Tick all 10 items in PROGRESS, advance PROGRESS focus to F19 candidate (AskUserQuestion with Previews per porting doc), run final verification:

```
cd HelixCode && make verify-compile
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \
  internal/render && echo BLUFF || echo clean
go test -count=1 ./internal/render/...
go test -count=1 -tags=integration ./tests/integration/...
go mod tidy
# go.mod expected change: golang.org/x/term v0.41.0 line drops
  "// indirect"
# go.sum expected change: NONE
git diff --exit-code go.sum # MUST be no-op
git diff go.mod             # one line: "// indirect" removed
  for golang.org/x/term
```

Commit chore(P1-F18-T10): close out feature 18 — no-flicker rendering. Push 4 remotes non-force (origin, helixdev, vasic-digital, gitlab per programme conventions). Request explicit user authorization at this step (CONST-043).

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types + interface (§3.3), T03 ansiRenderer (§4.2 + §4.3 + §5.2 dirty-region), T04 plainRenderer (§5.1 + §5.2 (c) zero-ANSI/zero-CR invariants + §5.4 ANSI pass-through), T05 viewport (§3.3), T06 factory (§4.1 + §4.4 truth table + CONST-042 source-scan), T07 streaming wire-in (§3.6 + §5.2 (a) + (b)), T08 tool-result wire-in (§4.3 + §3.3 RenderToolResult), T09 Challenge five phases (§5.2 + §6.3), T10 close-out (§9).
2. **TDD:** every code task starts with a failing test. Renderer impls test against `bytes.Buffer` (real `io.Writer`, no mocks). Factory uses `Options.IsTTY + Options.Env` constructor injection (test seam). Integration tests wire up the actual CLI struct with a `FakeProvider` to prove the production hot path goes through the renderer (anti-bluff §5.2 (a)).
3. **Type consistency:** `Renderer`, `Frame`, `RenderMode`, `Viewport`, `Options`, error sentinels (`ErrUnknownBlockID`, `ErrInvalidEnvMode`), env-var constants (`EnvRenderMode`, `ModePlainEnv`, `ModeFancyEnv`, `ModeAutoEnv`) — all match across spec §3.3 and plan T02–T08.
4. **Zero new external deps:** `stdlib` + existing `testify/zap` + `golang.org/x/term` (already in `go.sum` at v0.41.0; promoted from indirect to direct in T06). `go mod tidy` after T06 produces NO new entries in `go.sum` (only the indirect→direct promotion in `go.mod`). T10's verification step asserts this loudly.
5. **Anti-bluff (§5.2):** Challenge has FOUR always-run phases + 1 TTY-gated phase. Every phase records positive byte evidence (specific byte sequences, byte counts, signature presence). The four real-execution criteria — (a) production hot path goes through

- WriteToken/RenderFrame (T07 integration test asserts the today's "%S " pattern is ABSENT); (b) test observes the actual rendered bytes (Challenge captures via injected bytes.Buffer); (c) plainRenderer emits zero ANSI / zero CR (T04 invariant tests + Challenge phase 2 + 4); (d) dirty-region diff actually saves bytes (T03 + T05 + Challenge phase 3 with both positive AND negative-control assertions) — each have dedicated unit + integration + Challenge assertions. Byte-signature mismatch is a hard Challenge failure.
6. **CONST-042:** the renderer NEVER logs token text or frame content at any level. Source-scan test
(TestNoLoggerInfoOrDebugTakesContent_CONST042_SourceScan in T06) enforces this with a regex over internal/render/*.go. The Challenge's saved evidence file records byte lengths + signature presence, never the rendered text itself.
 7. **CONST-043:** stays on main, non-force to all four remotes; explicit user authorization is requested at T10 before pushing.
 8. **Custom ANSI vs library (Q1=A) — non-obvious call** (recorded in spec §2 trailer): no new deps + small render surface + applications/terminal_ui/ already uses tview/tcell for the full TUI; adding bubbletea here would be a third paradigm. The renderer is ~300 lines of pure Go; a library would be 50 KLOC.
 9. **Env var only, no slash, no cobra (Q5=B) — non-obvious call:** render mode is a startup property; switching mid-process would require viewport reset and unwinding in-flight in-place updates. v1 punts on the complexity. Slash command is a F18.5 candidate if user demand emerges.
 10. **fancy on a non-TTY is allowed (user opt-in) — non-obvious call** (recorded in spec §11 #1): when HELIXCODE_RENDER=fancy is explicitly set against a pipe/file, the user gets raw ANSI in their captured output. The default (auto) protects the 99% case.
 11. **plainRenderer strips \r from tokens silently — non-obvious call** (recorded in spec §11 #2): preserves log-grep-safety; documented; no toggle. Asymmetric vs ANSI pass-through (which IS preserved per §5.4) — \r corrupts log capture irrecoverably; ANSI is recoverable post-hoc with a sed.
 12. **Single-writer concurrency assumption — non-obvious call** (recorded in spec §5.3): the renderer's mutex is for safety, not for promised interleaving. Two concurrent Begin/WriteToken/Commit cycles for distinct block IDs serialise via mutex order (block A finishes, then block B) — NOT stream-aware merging. Acceptable v1 tradeoff; F18.5 may add a multiplexer.
 13. **Terminal width read once at factory time — non-obvious call** (recorded in spec §5.5 + §5.6): SIGWINCH handling deferred to F18.5; v1 documents that long lines are passed through and the terminal's own soft-wrap may leave artefacts on screen for in-place updates. Plain mode is unaffected.
 14. **No filesystem abstraction; pure os.Stdout + golang.org/x/term** — matches F17's discipline. One seam (Options.Writer + Options.IsTTY + Options.Env), not five.
 15. **Why RenderToolResult is a renderer-package helper, not registry-side** (recorded in spec §11 #11): keeps the registry decoupled from the renderer; the helper is called from CLI tool-result print sites in cmd/cli/main.go. Alternative (registry-side hook) would force every Tool.Execute consumer to know about the renderer. Helper is opt-in; existing callers that don't use it print results the old way (still works).